

MPI Parallelisation of the Mandelbrot Set

Jacob Gallop (Student ID: 20253130)

1 Introduction

The Mandelbrot set is defined as the set of complex numbers κ for which the iterative sequence

$$z_{n+1} = z_n^2 + \kappa, \quad z_0 = \kappa,$$

remains bounded. Since the set is contained within $|\kappa| < 2$, it suffices to iterate until $|z_n| > 2$ to classify a point as outside the set, with an upper bound of `MAXITER` = 1000 iterations. Increasing `MAXITER` computes a more accurate set of points, but increases the workload.

The serial implementation discretises the region $-2 \leq \text{Re}(\kappa) \leq 2$, $-2 \leq \text{Im}(\kappa) \leq 2$ on an $N \times N$ grid ($N = 15,000$) and records the iteration count at each pixel. The Mandelbrot set is symmetric about the real axis ($\text{Im}(\kappa) \rightarrow -\text{Im}(\kappa)$), so only the top half plus the middle row is computed for a total of $H = (N/2 + 1) \times N = 112,515,000$ pixels; the lower half is obtained by mirroring. The iteration counts are rendered and written to a PPM image (Figure 1).

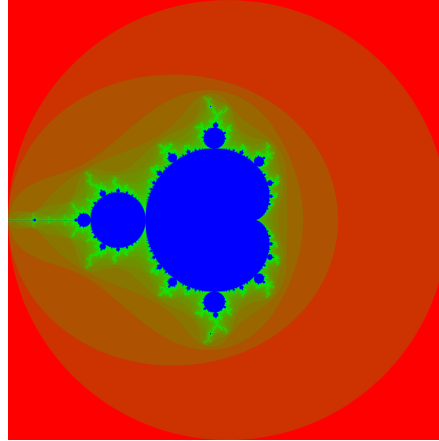


Figure 1: Mandelbrot set rendered by the serial code at $N = 4000$.

1.1 Serial Optimisation

After profiling the initial serial program (`serial_0`) with `gprof`, Table 1 revealed that 87.8% of execution time is spent on line 29:

```
29 while ((cabsf(z) <= 2.0f) && (k++ < MAXITER)) {
```

where complex magnitude `cabsf(z)` is computed as $|z| = \sqrt{\text{Re}(z)^2 + \text{Im}(z)^2}$. To avoid this expensive `sqrt()` operation entirely, the comparison is replaced with $(\text{Re}(z)^2 + \text{Im}(z)^2) > 4$, where points are computed in parametric form. This led to the following implementation, denoted as `serial_1`:

```
29 k = 1;
30 while (((zi * zi) + (zj * zj) <= 4.0f) && (k++ < MAXITER)) {
31     new_zi = (zi * zi) - (zj * zj) + ki;
32     zj = 2.0f * zi * zj + kj;
33     zi = new_zi;
34 }
```

which was later combined with the symmetry optimisation as `serial_2`.

1.2 Performance

Table 1 shows the percentage of execution time attributed to each source line in the hot loop.

Source Line	serial_0	serial_1	serial_2
Line 29: while condition (<code>cabsf</code>)	87.83%	—	—
Line 30: <code>z = z*z + kappa</code> / while condition	5.83%	73.58%	69.99%
Line 31: <code>new_zi</code> calculation	—	3.11%	6.18%
Line 32: <code>zj</code> update	0.16%	13.62%	15.45%
Line 36: <code>logf()</code> normalization	—	9.32%	7.64%
Total self time	12.43s	5.47s	2.75s

Table 1: gprof line-level profiling results ($N = 4000$)

Table 2 presents the hardware-level metrics collected via `perf stat`.

Metric	serial_0	serial_1	serial_2
Cycles (billions)	94.9	13.6	6.9
Instructions (billions)	89.9	28.4	14.3
IPC (instructions/cycle)	0.95	2.08	2.08
Wall time (seconds)	38.2	5.5	2.8
Speedup vs serial_0	1.0x	6.9x	13.6x

Table 2: perf stat hardware performance metrics ($N = 4000$)

The parametric implementation achieves a $6.9\times$ speedup, attributed to the reduced instruction count and increased CPU pipelining. Use of explicit arithmetic (vs `cabsf()`) allows the compiler to schedule simple floating-point math operations more efficiently, achieving an IPC increase of $\approx 2\times$. The addition of the symmetry optimisation in `serial_2` saw an expected speedup of $\approx 2\times$.

For comparison with MPI implementations, the final serial implementation (`serial_2`) completed with a wall-clock time of **35.632 s** using $N = 15,000$. All MPI benchmarks are conducted using $N = 15,000$ to remain consistent with the largest chunksizes of task 2.

2 Task 1: Static Decomposition (Block Partitioning)

2.1 Data Distribution and Algorithm

The H pixel indices are divided into $p = 10$ equal, contiguous blocks of size $C = H/p = 11,251,500$. Rank r receives and processes the block $[rC, (r+1)C)$. Because pixel indices are laid out row-by-row, each block corresponds to a horizontal strip of $\approx C/N \approx 750$ consecutive rows of the grid.

The root process initialises an array of sequential indices $0, 1, \dots, H-1$ and distributes them with `MPI_Scatter`. Each rank computes the iteration count for its assigned pixels using the standard escape-time loop, then all results are collected at the root via `MPI_Gather`. Barriers before and after the computation delimit the timing regions.

2.2 Key Code

```

1 int half = (N/2 + 1) * N;
2 int chunksize = half / n_ranks;
3
4 MPI_Scatter(n, chunksize, MPI_INT,
5            _n, chunksize, MPI_INT, 0, MPI_COMM_WORLD);
6
7 for (loop = 0; loop < chunksize; loop++) {
8     i = _n[loop] % N; j = _n[loop] / N;
9     ki = zi = (4.0f * (i - (float)N/2)) / N;
10    kj = zj = (4.0f * (j - (float)N/2)) / N;
11    k = 1;
12    while (((zi*zi)+(zj*zj) <= 4.0f) && (k++ < MAXITER)) {
13        new_zi = (zi*zi) - (zj*zj) + ki;
14        zj = 2.0f * zi * zj + kj; zi = new_zi;
15    }
16    _x[loop] = logf((float)k) * inv_log_maxiter;
17 }
18
19 MPI_Barrier(MPI_COMM_WORLD);
20 MPI_Gather(_x, chunksize, MPI_FLOAT,
21           x, chunksize, MPI_FLOAT, 0, MPI_COMM_WORLD);

```

2.3 Performance

	Wall-clock (s)	Speedup	Ideal ($f \approx 0$)
Serial	35.63	1.00×	—
Block ($p=10$)	17.21	2.07×	10×

Table 3: Task 1 performance ($N = 15,000$, 10 ranks).

By Amdahl’s law with negligible serial fraction ($f \approx 0$), the ideal speedup is $S = p = 10$. The measured speedup of $2.07\times$ falls far short, not because of serial overhead, but because of severe load imbalance.

2.4 Load Balancing

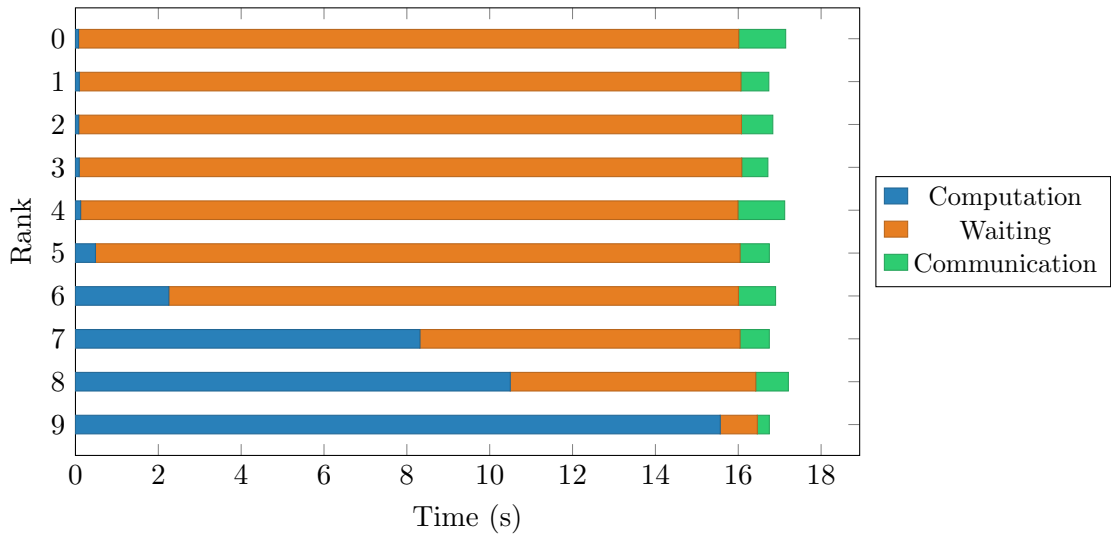


Figure 2: Per-rank timing breakdown for Task 1 (static block partitioning).

Figure 2 reveals extreme load imbalance. Rank 9, which processes the rows closest to the real axis (where the Mandelbrot set boundary lies), spends 15.6 s computing, while ranks 0–4, assigned to the top of the complex plane where most points escape quickly, compute for under 0.13 s—a ratio exceeding 200×.

The imbalance arises because the computational cost per pixel is highly skewed: pixels within the set boundary can require up to 1000 iterations, whereas pixels far from the set diverge in a handful. Block partitioning assigns contiguous rows to each rank, concentrating the expensive pixels on the ranks nearest the center. The resulting idle time (orange bars) dominates the wall-clock time, wasting the parallel resources.

3 Task 2: Dynamic Decomposition (Master–Worker)

3.1 Algorithm

To address the load imbalance of block partitioning, a master–worker scheme is used. Rank 0 acts as the master, distributing work to the $p-1 = 9$ workers in chunks of a tuneable `chunksize`:

1. The master sends each idle worker a starting index via `TAG_WORK`.
2. The worker computes the iteration counts for `chunksize` consecutive pixels, then returns the starting index and results via `TAG_RESULT`.
3. The master receives a result from *any* worker (`MPI_ANY_SOURCE`), stores it, and either sends the next chunk or a termination signal (`TAG_DONE`).
4. Workers loop until they receive `TAG_DONE`.

The implementation uses non-blocking communication (`MPI_Isend`, `MPI_Irecv`, `MPI_Waitall`) to overlap the send of results with the receipt of the next work assignment, reducing idle time.

Padding is applied so that $H + \text{pad}$ is divisible by `chunksize`; out-of-bounds pixels are computed but not written to the image.

3.2 Key Code — Master

```

1 idx = 0; n_busy = 0;
2 for (r = 1; r < n_ranks && idx < half; r++) {
3     MPI_Isend(&idx, 1, MPI_INT, r, TAG_WORK, comm, &req);
4     n_busy++; idx += chunksize;
5 }
6 while (idx < half || n_busy > 0) {
7     MPI_Recv(&result_idx, 1, MPI_INT, MPI_ANY_SOURCE,
8             TAG_RESULT, comm, &status);
9     src = status.MPI_SOURCE;
10    MPI_Irecv(&n[result_idx], chunksize, MPI_INT, src,
11             TAG_RESULT, comm, &req);
12    if (idx < half) {
13        MPI_Isend(&idx, 1, MPI_INT, src, TAG_WORK, comm, &req2);
14        idx += chunksize;
15    } else {
16        MPI_Isend(NULL, 0, MPI_BYTE, src, TAG_DONE, comm, &req2);
17        n_busy--;
18    }
19    MPI_Waitall(2, (MPI_Request[]){req, req2}, MPI_STATUSES_IGNORE);
20 }

```

3.3 Key Code — Worker

```

1 MPI_Recv(&idx, 1, MPI_INT, 0, MPI_ANY_TAG, comm, &status);
2 if (status.MPI_TAG == TAG_DONE) goto end;
3 while (1) {
4     MPI_Irecv(&next_idx, 1, MPI_INT, 0, MPI_ANY_TAG, comm, &req_next);
5     for (loop = 0; loop < chunksize; loop++) { /* compute */ }
6     MPI_Isend(&idx, 1, MPI_INT, 0, TAG_RESULT, comm, &req1);
7     MPI_Isend(_n, chunksize, MPI_INT, 0, TAG_RESULT, comm, &req2);
8     MPI_Waitall(3, (MPI_Request[]){req_next, req1, req2}, statuses);
9     if (statuses[0].MPI_TAG == TAG_DONE) break;
10    idx = next_idx;
11 }

```

3.4 Load Balancing at Chunksize $\approx 100,000$

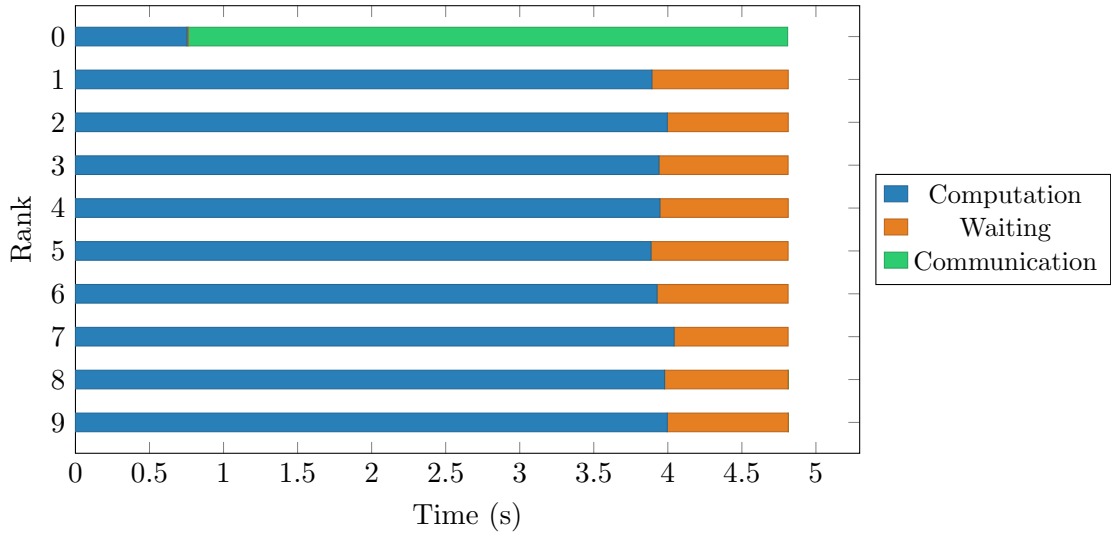


Figure 3: Per-rank timing for Task 2 (chunksize $\approx 100,000$, 10 MPI processes).

Figure 3 shows a dramatic improvement over Task 1. All nine workers spend 3.89–4.04 s computing (a spread of only $\pm 2\%$), with negligible communication overhead. Workers that finish a cheap chunk quickly are immediately assigned another, so the workload self-balances over many scheduling rounds. The master (rank 0) performs little computation itself but spends 4.05 s on communication managing the work distribution. The total wall-clock time is 4.81 s.

3.5 Runtime vs. Chunksize

Figure 4 exhibits three regimes:

- **Small chunksize** (< 100): Communication overhead dominates. Each chunk requires a round-trip of MPI messages, so $\sim H/C$ message exchanges occur. At chunksize 2, the runtime exceeds even the serial baseline.
- **Optimal range** (~ 500 – $100,000$): The runtime plateaus near 4.75 s (speedup $\approx 7.5\times$). Communication cost per chunk is amortised over enough computation, and chunks are small enough that dynamic scheduling balances the load effectively. The optimum is broadly flat, making the method robust to the choice of chunksize within this range.
- **Large chunksize** ($> 10^6$): With fewer, bigger chunks the scheduling degenerates toward static block partitioning. Load imbalance re-emerges and the runtime climbs steeply. At $C = H \approx 1.1 \times 10^8$, a single worker performs all the work and the runtime approaches the serial time.

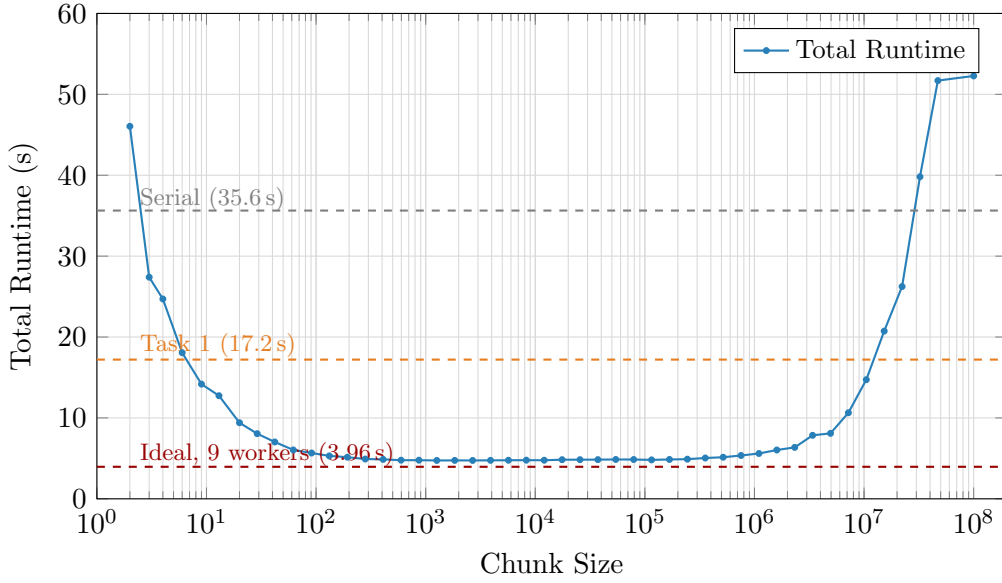


Figure 4: Total runtime vs. chunksize for Task 2 with 9 workers. Reference lines show the serial time, ideal parallel time, and Task 1 block-partitioning time.

3.6 Comparison with Task 1

When the chunksize equals one-tenth of the total work ($H/10 = 11,251,500$), each of the 9 workers receives a large, contiguous block. The data at $C = 10,481,131$ gives a wall-clock time of 14.7s, comparable to Task 1’s 17.2s. The per-rank profile at this chunksize (not shown) exhibits the same pattern as Task 1: ranks assigned to the centre of the complex plane compute for ~ 14 s while ranks at the edges finish in under 0.1s. This confirms that large-chunksize dynamic scheduling recovers the load-imbalanced behaviour of block partitioning.

The small discrepancy (14.7s vs. 17.2s) arises because the fastest worker finishes its first chunk quickly and receives a second one, providing a degree of dynamic rebalancing that pure block partitioning cannot.

4 Task 3: Static Decomposition (Cyclic Partitioning)

4.1 Data Distribution and Algorithm

Cyclic (round-robin) partitioning assigns pixel i to rank $(i \bmod p)$. This shuffles the workload across all rows of the grid so that each rank processes a mix of cheap and expensive pixels, naturally balancing the load.

The implementation constructs a shuffled index array on the root:

```

1 for (idx = 0; idx < chunksize; idx++)
2     for (blk = 0; blk < n_ranks; blk++)
3         n[c++] = blk * chunksize + idx;

```

`MPI_Scatter` distributes C consecutive elements from this array to each rank, so rank r receives the indices $\{rC, rC+1, \dots\}$ in cyclic order, which correspond to every p -th pixel in the original domain.

After computation and `MPI_Gather`, the root reverses pixel iterations into their original order, and computes pixel colours:

```

1 for (idx = 0; idx < chunksize; idx++)
2     for (blk = 0; blk < n_ranks; blk++) {
3         loop = blk * chunksize + idx;
4         x[loop] = logf((float)n[c++]) * inv_log_maxiter;
5     }

```

4.2 Performance

	Wall-clock (s)	Speedup	Ideal ($f \approx 0$)
Block (Task 1)	17.21	$2.07\times$	$10\times$
Cyclic (Task 3)	4.46	$7.99\times$	$10\times$

Table 4: Comparison of static block and cyclic partitioning.

4.3 Load Balancing

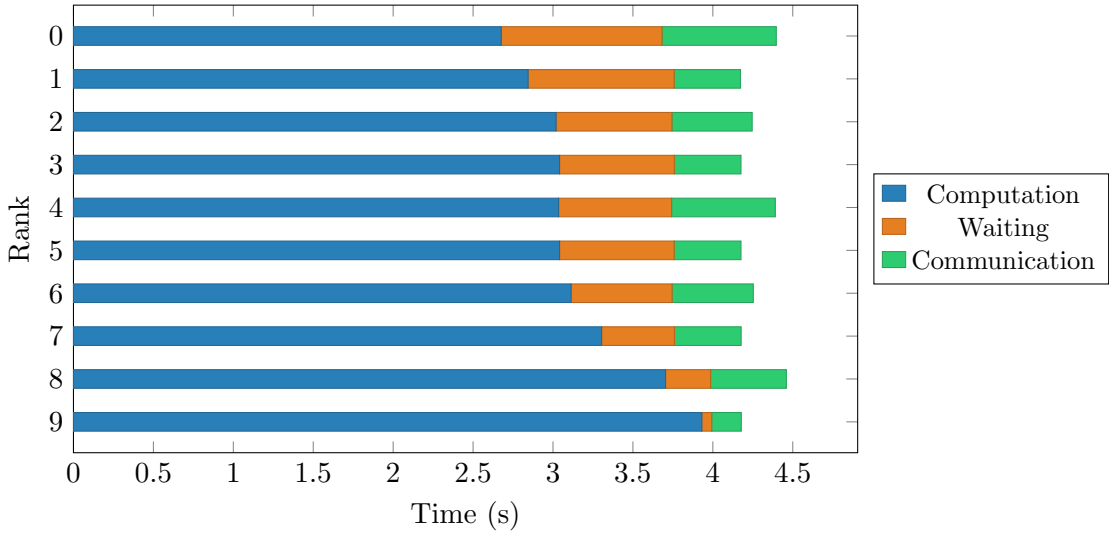


Figure 5: Per-rank timing breakdown for Task 3 (static cyclic partitioning).

4.4 Comparison with Block Partitioning

Figure 5 shows a more balanced workload than Task 1 (Figure 2). Computation times range from 2.68 s (rank 0) to 3.93 s (rank 9)—a ratio of just $1.47\times$, compared to over $200\times$ for block partitioning.

A residual gradient remains because the cyclic partitioning is not perfectly uniform: rank 9 receives the last pixel from each “round,” which systematically biases it toward rows closer to the set boundary. However, this effect is minor and the waiting times (orange bars) are much shorter than in Task 1.

The speedup improves from $2.07\times$ (block) to $7.99\times$ (cyclic) with the same communication pattern (one `Scatter` + one `Gather`). The improvement is entirely due to better load distribution.

5 Conclusions

Block partitioning is the simplest to implement (one `Scatter` + one `Gather`) but performs poorly on workloads with spatial non-uniformity, as is the case for the Mandelbrot set. Its speedup of $2.07\times$ with 10 processes represents a parallel efficiency of only 21%.

Dynamic scheduling achieves near-ideal load balance (work-time spread $< 4\%$) and a speedup of $7.49\times$. Its robustness to workload irregularity makes it the most generally applicable strategy. The cost is added code complexity and the dedication of one process to scheduling. The optimal chunksize lies in a broad plateau (~ 500 – $100,000$), making it easy to tune. For very small chunks communication overhead dominates; for very large chunks load imbalance re-emerges.

Method	Runtime (s)	Speedup	Work Ratio	Efficiency
Serial	35.63	—	—	—
Block (Task 1)	17.21	2.07×	207×	20.7%
Dynamic (Task 2)	4.76	7.49×	1.04×	83.3%*
Cyclic (Task 3)	4.46	7.99×	1.47×	79.9%

* Efficiency relative to 9 computing workers. Relative to all 10 processes: 74.9%.

Table 5: Summary comparison of all three methods at $N = 15,000$. Work Ratio = $\max(\text{work}) / \min(\text{work})$ across worker ranks.

Cyclic partitioning delivers the highest speedup (7.99×) with the same simplicity as block partitioning. It succeeds because the Mandelbrot workload varies *smoothly* across the grid; interleaving ensures each rank samples all regions evenly. Unlike dynamic scheduling, all 10 processes contribute to computation.

Recommendation. For the Mandelbrot problem, **cyclic partitioning is the best strategy**: it combines the highest speedup, lowest implementation complexity, and minimal communication overhead. Dynamic scheduling would be preferable for workloads with less predictable structure, where static partitioning cannot guarantee balance.